Event Generation with Parallel Langevin Sampling and Learned Stein Diagnostics

Rob Verheyen*
Department of Physics and Astronomy, University College London, London, WC1E 6BT, UK

Abstract

Efficient event generation is a major computational challenge for precision collider phenomenology, especially for high-multiplicity final states where matrix-element evaluations are expensive and rejection-sampling efficiencies are low. We study an alternative approach based on many parallel underdamped Langevin chains, retaining one terminal state from each chain to obtain unweighted events while avoiding within-chain autocorrelation. A learned Stein discrepancy is used as a convergence diagnostic, providing a data-driven estimate of the relaxation time. We apply the method to tree-level $u\bar{u} \to Z + ng$ event generation and find that relaxation requires only a modest number of exact-target Langevin steps, with mild growth over the multiplicities studied. Finally, we show that simple neural-network surrogate initialization can substantially reduce the required number of exact matrix-element and gradient evaluations.

1 Introduction

The upcoming high-luminosity upgrade to the Large Hadron Collider (LHC) [1] and its potential successors [2, 3] are slated to achieve unprecedented experimental precision. The theory side must match this precision, not only in systematic control but also in statistical accuracy. With current technical implementations, the latter, in the form of the required number of Monte Carlo (MC) event samples, is expected to become the dominant computational bottleneck [4–8].

The most expensive component of the MC event generator chain [9] is the sampling of the hard proton-proton collision differential cross section

$$\mathrm{d}\sigma_{pp\to X_n} = \frac{1}{\mathcal{F}} \sum_{i,j} f_i(x_1)\, f_j(x_2)\, |M_{ij\to X_n}(\Phi_n)|^2 \,\mathrm{d}x_1\,\mathrm{d}x_2\,\mathrm{d}\Phi_n, \tag{1}$$

where the sum runs over the constituents of the proton (quarks, antiquarks and gluons), f are their corresponding parton density functions, $|M_{ij\to X_n}|^2$ is the squared matrix element, and $\mathcal{F}$ is the incoming partonic flux factor. The integration measure is given by the two partonic momentum fractions $\mathrm{d}x_{1,2} \in [0, 1]$ and the n-body phase-space measure $\mathrm{d}\Phi_n$, which integrates over the final-state momenta while ensuring momentum conservation. The total sampling dimensionality of this object is $3n - 2$.

For large multiplicities n and for high theoretical precision, the evaluation of the matrix element can be prohibitively expensive. The method used to sample Eq. (1) must therefore be as efficient as possible in terms of evaluations, even if this comes at increased algorithmic cost. Current methods are still largely based on the VEGAS algorithm [10, 11] with many physics-inspired improvements [12–19]. These methods all rely on an adaptive approximation of an unnormalized target density $\widetilde{p}(x)$ (the differential cross section) by some density $q(x)$ that is

*robverheyen@gmail.com

easily sampled. One can then sample from $q(x)$ and assign weights $w(x) = \widetilde{p}(x)/q(x)$. To obtain unweighted samples, rejection sampling must be performed with acceptance probability

$$p_{\texttt{acc}}(x) = \frac{w(x)}{\max_x w(x)}. \tag{2}$$

The unweighting efficiency $\epsilon = \mathbb{E}_{x\sim q}\left[p_{\texttt{acc}}(x)\right]$ then determines the required number of matrix element evaluations per generated sample. Eq. 2 is very sensitive to the value of $\max_x w(x)$, which may originate from tiny regions of phase space. Thus, to improve efficiency, one may make use of a reduced maximum and leave some undesirable weighted events [20]. Nevertheless, for increasing multiplicities, the unweighting efficiency can still degrade by several orders of magnitude [21].

Recent work has used modern machine-learning (ML) architectures such as normalizing flows in place of VEGAS to improve efficiencies [22–29]. These approaches are however still ultimately based on rejection sampling and thus inherit its disadvantages. In this work, we propose a different approach based on parallelized Markov chain Monte Carlo (MCMC) supplemented by an ML-based convergence diagnostic. MCMC methods have previously been explored for hard-process phase-space sampling and event generation [30–33]. Our method places particular emphasis on avoiding within-chain autocorrelations and on reaching MCMC stationarity, i.e. samples being representative of $\widetilde{p}(x)$. Furthermore, we investigate how neural network surrogates of the target distribution can further reduce the required number of matrix-element evaluations [34–36].[1]

2 Method

We now describe our sampling construction, targeting an unnormalized density $\widetilde{p}(x)$ on the unit hypercube $x \in [0,1]^d$. To avoid within-chain autocorrelation, we evolve many Markov chains in parallel on hardware accelerators and retain only the terminal state from each chain. The learned Stein discrepancy introduced below is used to determine the relaxation time t_{rel}, i.e. the number of MCMC steps after which the ensemble is representative of the target.

2.1 Underdamped Langevin Algorithm

Our MCMC framework is designed for rapid convergence under two practical constraints: the sampler should be straightforward to run over many parallel chains, and it should use expensive target-density and gradient evaluations sparingly. Sophisticated Hamiltonian Monte Carlo (HMC) methods [37] like NUTS [38] implement adaptive trajectory lengths which leads to complex control flow and thus difficult parallelizability. Moreover, HMC typically requires several intermediate gradient evaluations before a single Metropolis-Hastings (MH) correction is applied. We therefore use an Underdamped Langevin Algorithm (ULA) [39, 40], which retains momentum between steps while applying an MH correction after each proposal.

Each MCMC chain carries a position x and a velocity v. We assume we have access to the score $s(x) = \nabla \log \widetilde{p}(x)$. For stability purposes the score is regularized component-wise to $\widetilde{s}_i(x) = s_i(x)/(1 + \eta_i |s_i(x)|)$, where η_i are the MCMC step sizes [41]. At time step t, a new proposal x', v' is generated following the leapfrog step

$$\begin{aligned} v_{1/2} &= v_t + \frac{1}{2}\Lambda_\eta\, \widetilde{s}(x_t), \\ x' &= x_t + \Lambda_\eta\, v_{1/2}, \\ v' &= v_{1/2} + \frac{1}{2}\Lambda_\eta\, \widetilde{s}(x'), \end{aligned} \tag{3}$$

[1] The code used in this study is available at https://github.com/rbvh/ula_stein_evt_gen.

where $\Lambda_\eta = \mathrm{diag}(\eta_1, \ldots, \eta_d)$. This step discretizes the Hamiltonian dynamics on the augmented density

$$\pi(x, v) \propto \widetilde{p}(x) \exp\left(-\frac{1}{2} v^T v\right). \tag{4}$$

In the continuous-time limit, the Hamiltonian flow conserves the Hamiltonian $H(x, v) = -\log \widetilde{p}(x) + v^T v/2$ and preserves phase-space volume. As a result, $\pi(x, v)$ as well as its x-marginal, which is proportional to the desired target density $\widetilde{p}(x)$, is invariant. However, at finite step sizes an integration error is introduced. By applying a MH acceptance step with probability

$$a = \min\left\{1, \exp\left[-H(x', v') + H(x_t, v_t)\right]\right\} \tag{5}$$

the discretization bias is corrected. Upon rejection, the position is kept and the incoming momentum is reversed $v_t \to -v_t$. After the MH step, the momentum is refreshed

$$v_{t+1} = \beta v_{\mathrm{acc}} + \sqrt{1 - \beta^2}\, \zeta, \qquad \zeta \sim \mathcal{N}(0, I). \tag{6}$$

These momentum updates preserve the Gaussian velocity marginal and inject randomness into the otherwise deterministic Hamiltonian evolution, allowing the chain to move between trajectories and explore the full target distribution. After equilibration, terminal positions drawn from these chains are distributed according to $\widetilde{p}(x)$.

We write the step sizes as $\log \eta_i = \log \epsilon + \log \rho_i$, with $\sum_i \log \rho_i = 0$. The scalar scale ϵ is initialized by doubling or halving until the average acceptance probability is close to one half [38], while the directions ρ_i are initialized from the diagonal score second moment, motivated by Fisher preconditioning [42, 43],

$$F_{ii,t} = \mathbb{E}_{x_t}\left[s_i(x_t)^2\right], \qquad \log \rho_i = -\frac{1}{2} \log F_{ii,t} + \frac{1}{2d} \sum_{j=1}^{d} \log F_{jj,t}. \tag{7}$$

This fixes $\prod_i \rho_i = 1$ and increases step sizes in directions with smaller typical score variation. During the chain, $\log \epsilon$ is adapted by ascending the expected squared jump distance $\mathrm{ESJD}(\epsilon) = \mathbb{E}_{x_t, v_t, \zeta}\left[a \|x' - x_t\|^2\right]$ at fixed ρ, a standard proxy for MCMC efficiency [44, 45]. The directions are adapted by updating ρ_i with new estimates of the diagonal score second moment. Both the ESJD scale update and the score-moment direction update use the same diminishing Robbins–Monro schedule, so that the transition-kernel changes vanish asymptotically, as required for stationarity [46, 47]. All expectations are evaluated over the parallel chain ensemble.

2.2 Mirror Langevin Dynamics

After the application of a mapping, the phase-space coordinates used for the cross-section integrand live on the bounded domain $x \in [0, 1]^d$, while ULA is most naturally formulated on $\mathbb{R}^d$. We therefore perform the MCMC evolution in mirror space [48], in which Langevin dynamics evaluated through a suitable mirror map retain the convergence properties of the corresponding unconstrained algorithm. We use the element-wise transform $y_i = \Phi^{-1}(x_i) = \sqrt{2}\, \mathrm{erf}^{-1}(2x_i - 1)$. The mirrored target density is the corresponding change of variables

$$\log \widetilde{p}_{\mathrm{mir}}(y) = \log \widetilde{p}(\Phi(y)) - \frac{1}{2}\|y\|^2 + \mathrm{const}. \tag{8}$$

Note that the constant drops out in the score.

2.3 Learned Stein Discrepancy

To monitor convergence of the ULA chains, we need a distributional discrepancy that can be evaluated using samples from the chain and the unnormalized density of the target. The Stein

discrepancy provides such a diagnostic [49, 50]. For a vector-valued critic function $f : \mathbb{R}^d \to \mathbb{R}^d$, the Stein discrepancy between a smooth, sampled distribution p and a smooth target distribution q is

$$S(p,q) = \sup_{f\in\mathcal{F}} \mathbb{E}_{x\sim p}\left[s_q(x)^T f(x) + \nabla\cdot f(x)\right], \tag{9}$$

where $\mathcal{F}$ is a norm-bounded class of differentiable functions for which the expectation over p is finite and $q(x)f(x)$ has vanishing tails. For sufficiently rich critic classes, this quantity is non-negative and satisfies $S(p,q) = 0$ if and only if $p = q$ [51].

Kernel Stein discrepancies are a popular choice for $\mathcal{F}$ [52–54], but their standard estimators involve pairwise sums that scale as n^2 in the number of samples n. Since we typically work in a regime with large numbers of samples, n^2 evaluation is infeasible, and a more natural choice would be one that exploits this large data size. We thus use the learned Stein discrepancy (LSD) [55, 56], taking $\mathcal{F}$ to be a class of square-integrable differentiable functions that satisfy the Stein boundary condition, and representing the critic by a neural network f_ϕ. In practice, square integrability is enforced by adding a quadratic regularization term to the training objective,

$$\mathcal{L}_\lambda(\phi;p,q) = -\mathbb{E}_{x\sim p}\left[s_q(x)^T f_\phi(x) + \nabla\cdot f_\phi(x) - \lambda\|f_\phi(x)\|^2\right]. \tag{10}$$

After training, the learned Stein discrepancy is evaluated with the learned critic on a held-out sample, without the regularization term.

2.4 Sharp Boundaries

Physical cross sections typically involve sharp phase-space cuts, which restrict the target density to an allowed region with a hard boundary. The derivation of Eq. 9 involves an integration by parts over this region, and a boundary contribution will thus remain unless the Stein test function has vanishing flux through the cut surface [57, 58]. We address this issue by introducing a scalar boundary function $h(x)$ that is positive inside the allowed region and vanishes on the cut boundary. The Stein operator is then applied to the vector field hf rather than to f itself [57, 59],

$$S_h(p,q) = \sup_{f\in\mathcal{F}} \mathbb{E}_{x\sim p}\left[h(x)s_q(x)^T f(x) + h(x)\nabla\cdot f(x) + \nabla h(x)^T f(x)\right]. \tag{11}$$

2.5 Surrogates

The sampling strategy described above relies on gradients of the target density. Differentiable matrix element implementations have recently become available [29, 60–63], enabling the sampling strategy set out above, but at higher computational expense. This motivates us to consider the use of learned surrogates to reduce the required number of evaluations.

Neural surrogates for scattering amplitudes and matrix elements have been studied extensively [34–36, 64–78]. In the present setting, one can train a surrogate on a relatively limited set of points without requiring gradients. Then, chains can be evolved cheaply through the early part of the trajectory using the surrogate as target, before switching over to a shorter sequence of ULA steps using the exact target. The surrogate is thus only used to accelerate equilibration, while the final evolution remains tied to the physical target density. We investigate this strategy in Section 4.

3 Validation

Before applying the method to collider cross sections, we validate the Stein diagnostic on a target for which the MCMC convergence can be computed analytically. Starting from a fixed

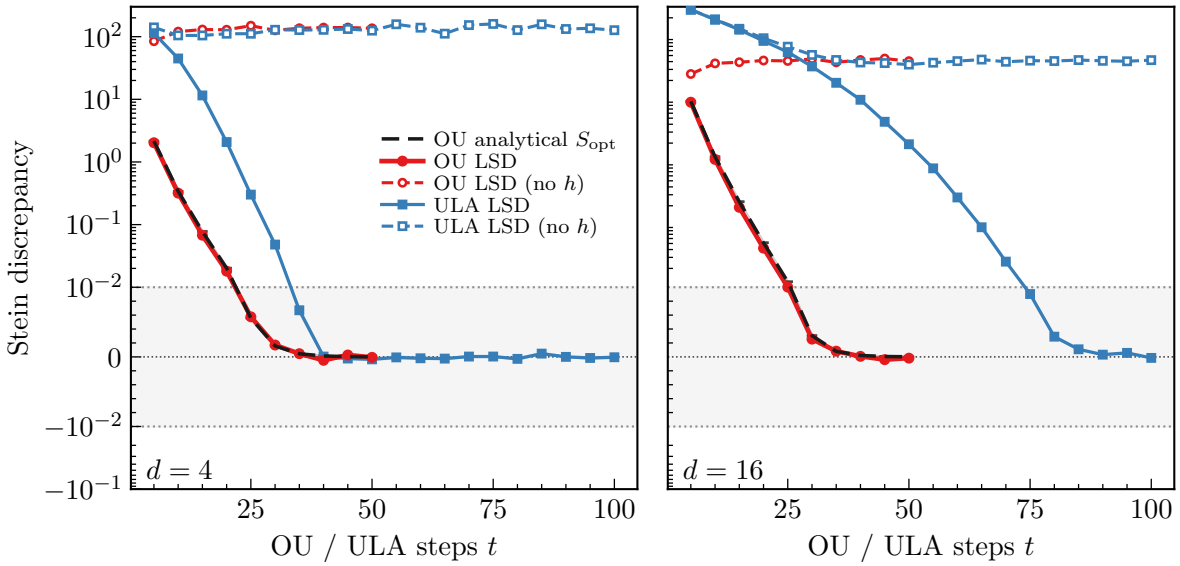


Figure 1: The LSD as a function of the number of steps for the exact OU reference process with $\eta = 0.5$ (red) and the underdamped Langevin algorithm (ULA, blue). Dashed lines show the boundary-function ablation. The black line shows the analytic optimal Stein discrepancy S_{opt}.

point μ_0 away from some target Gaussian mean μ_q and diagonal target Σ_q, we sample an exact Ornstein–Uhlenbeck (OU) transition kernel

$$x_{t+1} = \mu_q + e^{-\eta\Sigma_q^{-1}}(x_t - \mu_q) + \Sigma_q^{1/2}\left(I - e^{-2\eta\Sigma_q^{-1}}\right)^{1/2}\xi, \qquad \xi \sim \mathcal{N}(0, I), \tag{12}$$

where η is the step size. After t steps, the marginal distribution is Gaussian with

$$\begin{aligned} \mu_t &= \mu_q + e^{-t\eta\Sigma_q^{-1}}(\mu_0 - \mu_q) \\ \Sigma_t &= \Sigma_q\left(I - e^{-2t\eta\Sigma_q^{-1}}\right). \end{aligned} \tag{13}$$

Thus p_t converges to the target density as $t \to \infty$, yielding a controlled and testable sequence of distributions. To validate the role of the boundary function, we apply a radial cut that removes 20% of the target probability mass centered around the target mean μ_q.

Under Eq. (10), the optimal Stein discrepancy is [55]

$$S_{\text{opt}}(p_t, q) = \frac{1}{2\lambda}\mathbb{E}_{x\sim p_t}\left[\|s_q(x) - s_{p_t}(x)\|^2\right]. \tag{14}$$

We compare this optimal value against the LSD computed on OU samples and on ULA samples. The validation results use dimensions $d = 4$ and $d = 16$, with target parameters

$$\mu_q = \text{linspace}(-2, 2, d), \qquad \Sigma_q = \text{diag}\Big(\text{linspace}(0.5, 2, d)^2\Big). \tag{15}$$

The OU process is initialized from the fixed point $\mu_0 = \mu_q + 3$, while ULA is initialized from a unit Gaussian. The LSD regularization is $\lambda = 0.1$. The Stein critic is modelled with a five-layer residual GeLU network with hidden width 128. Optimization is performed with the Adam optimizer [79] on $n = 10^7$ chains, and final LSD values are obtained using 10-fold cross validation. The boundary function is $h(x) = \max\{1 - r_{\text{cut}}^2/r^2(x), 0\}$.

The validation results are shown in Fig. 1. For the exact OU samples, the learned LSD tracks the analytic value S_{opt}, supporting its use as a quantitative convergence diagnostic. The ULA curves show similar monotonic decay towards zero, with the higher-dimensional case converging more slowly, qualitatively consistent with the expected $\sqrt{d}$ scaling of Metropolis-adjusted Langevin methods [80]. In the absence of the boundary function, the remaining finite boundary contribution prevents the Stein discrepancy from approaching zero.

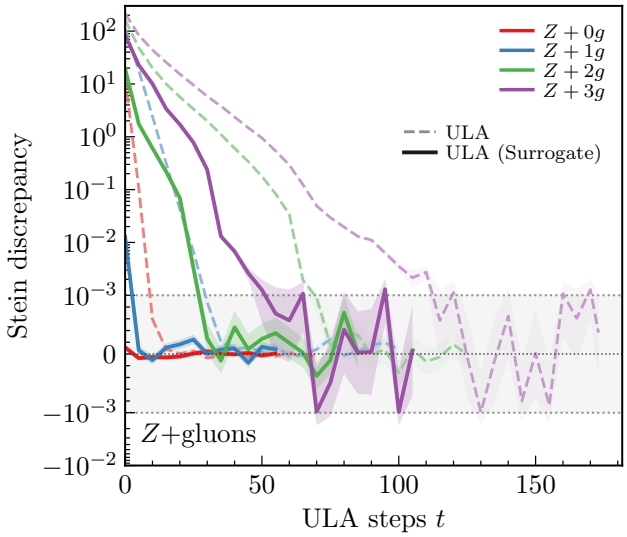


Figure 2: The LSD as a function of the number of ULA steps for $Z + ng$ cross sections. Colours denote gluon multiplicities, while line styles distinguish ULA with and without surrogate initialization.

4 Results

We now apply ULA and the LSD diagnostic to sample the cross section for the process $u\bar{u} \to Z + ng$. Matrix elements and their gradients are evaluated using MadJax [60] with several modifications for numerical stability. We use the NNPDF4.0 LO set [81], and implement the PDF and α_s interpolation in JAX [82] following the LHAPDF interpolation prescription [83]. We set $\sqrt{s} = 13\,\text{TeV}$ and apply cuts $p_\text{T} > 20\,\text{GeV}$ ($n \geq 1$) and $|y| < 5$ to all final-state particles and a separation cut $\Delta R > 0.4$ to the gluons. Phase space is mapped to the $[0,1]^{3n+1}$ hypercube using the Chili [84] vector boson plus jets map. The Stein diagnostic uses the same critic architecture and optimization strategy as in the OU validation, with $\lambda = 0.1$. The boundary function is

$$h(x) = \left[\frac{1}{2+\binom{n}{2}}\left(\frac{1}{1-x_a}+\frac{1}{1-x_b}+\sum_{1\leq i<j\leq n}\frac{1}{1-\Delta R_\text{cut}^2/\Delta R_{ij}^2}\right)\right]^{-1}, \tag{16}$$

for points that pass the cuts, and 0 for points that do not.

We use ULA and the LSD diagnostic to generate 10^7 samples for $n = 0, 1, 2, 3$, both with and without neural surrogates. The surrogate is modelled with the same architecture as the Stein critic. It is optimized in log space with the Adam optimizer and a mean-squared error loss using only 10^6 samples. We then run ULA on the surrogate for a large number of steps. The terminal states are then used as the initial states for the ULA run, which is otherwise initialized with unit Gaussian samples. We define the relaxation time t_rel as the first checkpoint for which the learned LSD is no more than one standard error above zero, $S_\text{LSD} \leq \Delta S_\text{LSD}$. After the trigger, we evolve the chain for a further $\max(0.5\,t_\text{rel}, 50)$ ULA steps. The convergence results are shown in Fig. 2, and relaxation-time triggers are summarized alongside VEGAS unweighting results in Table 1. The surrogate warm start substantially reduces the number of exact-target ULA steps required for relaxation.

We compare with VEGAS, reporting $1/\epsilon$, i.e. the number of weighted events per unweighted event. Following standard practice, we also report $1/\epsilon_{0.001}$ and $1/\epsilon_{0.01}$, in which overweights contributing the quoted fractions to the total integral are allowed to remain.

We observe that the VEGAS proposal requirement increases substantially with gluon multiplicity, while Langevin relaxation time grows much more mildly. Allowing residual weights

	ULA t_{rel}		VEGAS		
Process	Standard	Surrogate	$1/\epsilon$	$1/\epsilon_{0.001}$	$1/\epsilon_{0.01}$
$Z+0g$	20	5	$2.69 \pm 5.5 \times 10^{-5}$	$2.53 \pm 8.2 \times 10^{-4}$	$1.25 \pm 2.5 \times 10^{-4}$
$Z+1g$	50	5	20.0 ± 0.49	8.75 ± 0.029	$4.31 \pm 2.2 \times 10^{-3}$
$Z+2g$	75	35	540 ± 140	74.8 ± 2.3	30.0 ± 0.085
$Z+3g$	115	55	1450 ± 240	460 ± 23	148 ± 1.3

Table 1: Exact-target ULA relaxation-time checkpoints and VEGAS inverse unweighting efficiencies for the same cross sections. To estimate the substantial uncertainties, errors are computed for the VEGAS results over 10 runs.

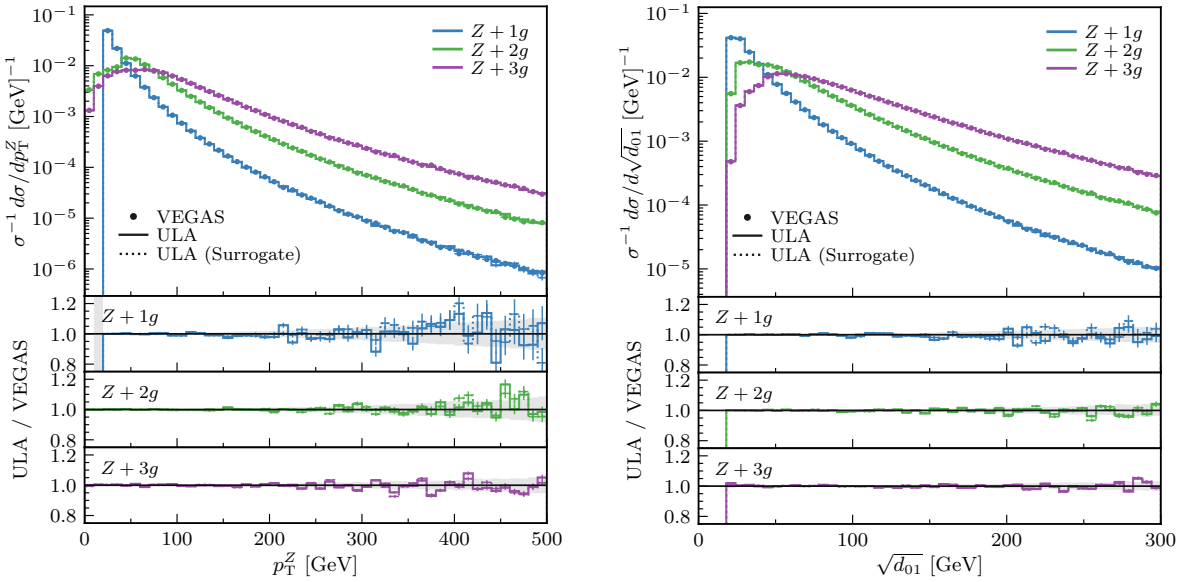


Figure 3: Normalized differential distributions for the transverse momentum of the Z boson and the exclusive k_t clustering scale $\sqrt{d_{01}}$. Filled markers show the VEGAS reference, solid lines show ULA samples, and dotted lines show ULA samples after surrogate initialization. The lower panels show the corresponding ratios to VEGAS for each multiplicity.

improves efficiencies considerably, but leaves some events with non-unit weights, whereas the MCMC construction directly produces unit-weight events. We note that methods like VEGAS benefit greatly from techniques like multichanneling [12], and a more comprehensive comparison should be performed in the context of a full event generator.

To confirm convergence of the ULA chains, we show the normalized distributions of several nontrivial physical observables in Figures 3 and 4. We validate ULA results with and without surrogate initialization against independent VEGAS estimates. Figure 3 shows the vector-boson transverse momentum p_{T}^{Z} and the first jet-resolution scale $\sqrt{d_{01}}$, obtained by exclusive k_t clustering with radius parameter $R = 1$ [85, 86]. Both observables start at $Z + 1g$ and their distributions span multiple orders of magnitude. Figure 4 shows the longitudinal boost rapidity of the full final-state system, $\frac{1}{2}\log(x_a/x_b)$, and the largest absolute gluon rapidity, $\max_i |y^{j_i}|$. In all cases, the terminal ULA samples reproduce the VEGAS distributions within statistical uncertainties.

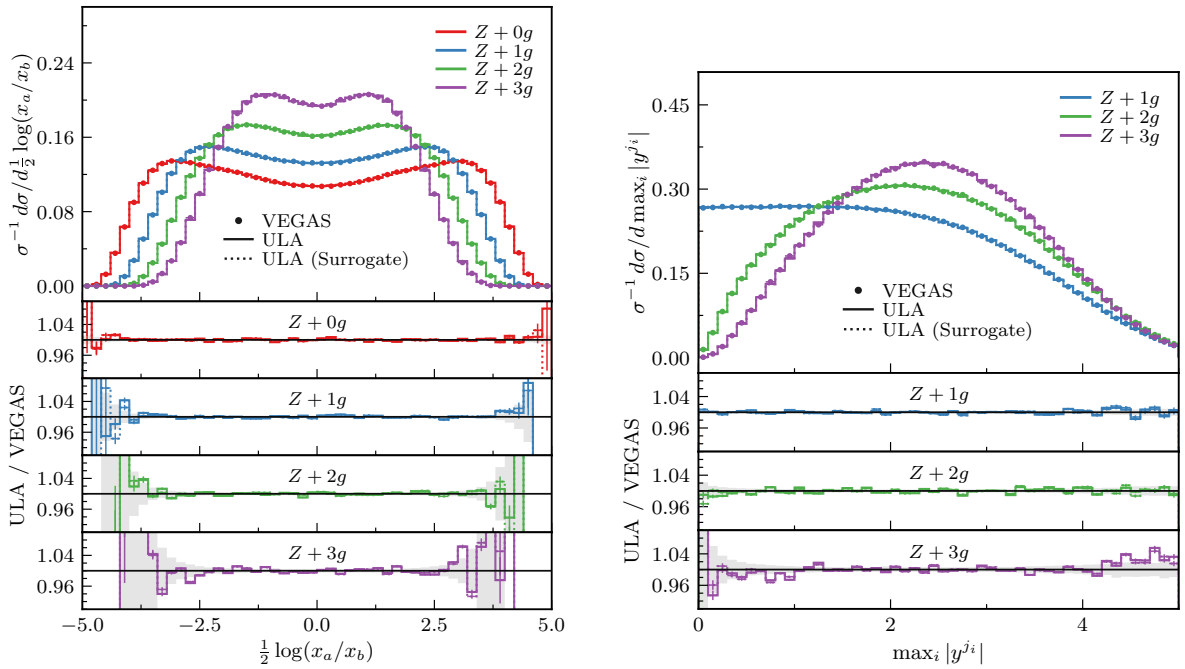


Figure 4: Normalized differential distributions for the rapidity of the full final-state system, $\frac{1}{2}\log(x_a/x_b)$, and the largest absolute gluon rapidity, $\max_i |y^{j_i}|$. Filled markers show the VEGAS reference, solid lines show ULA samples, and dotted lines show ULA samples after surrogate initialization. The lower panels show the corresponding ratios to VEGAS for each multiplicity.

5 Discussion

The results above suggest that gradient-based parallel MCMC is a practically viable alternative to rejection-based event generation, at least in the regime studied here. The central practical quantity is the number of exact matrix-element evaluations needed before terminal samples can be used. For the $Z + ng$ examples, the learned Stein diagnostic triggers after a modest number of exact-target ULA steps, and the increase with multiplicity is mild over the range considered. This is encouraging because unweighting efficiencies can degrade by orders of magnitude as the final-state multiplicity increases [34, 35]. Furthermore, the methods explored here generate strictly unweighted events, while in the usual rejection sampling setup residual weights may remain when using the maximum weight would otherwise reduce the efficiency even further. The increased algorithmic cost of our method is negligible when compared with the compute required for the matrix-element evaluations. Like conventional rejection-based generation, our method is also naturally parallel: each chain can be evolved independently, while the adaptation and Stein diagnostic require only ensemble-level reductions. This makes the method well-matched to GPU/TPU execution.

The required number of evaluations can be further reduced through the use of surrogate initialization. In our implementation the surrogate is deliberately simple: it uses the same network architecture as the Stein critic and is trained on a comparatively small set of uniformly sampled phase-space points. Nevertheless, it substantially shortens the exact-target relaxation time. This should be viewed as a conservative demonstration rather than an optimized surrogate strategy. More expressive architectures, improved training samples, or active refinement in the regions explored by the chains could plausibly make the warm start closer to the physical target and further reduce the number of exact matrix-element and gradient evaluations.

It is worth noting that the additional computational cost of gradient evaluation is relatively mild. Reverse-mode automatic differentiation requires only a single forward and backward

pass [87, 88], regardless of the input dimensionality. At least for tree-level squared matrix elements, which are composed of rational functions of Lorentz-invariant (pseudo)scalars, the computational cost of gradient evaluation should therefore approach only a small constant factor over the value evaluation with sufficiently efficient implementations.

The present study considers a fixed partonic channel, tree-level matrix elements, and a simple phase-space map. The observed scaling, and more generally any comparison with current methods, can only be tested in the context of an implementation in full event generator programs. Still, the combination of parallel chains, an explicit relaxation-time diagnostic, and surrogate initialization provides a concrete route towards event generation in which the expensive exact target is evaluated as few times as possible.

Acknowledgements

This work was supported by the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (grant agreement No. 788223, PanScales).

References

[1] I. Zurbano Fernandez et al., *High-Luminosity Large Hadron Collider (HL-LHC): Technical design report*, no. 10/2020 in CERN Yellow Reports: Monographs, CERN, Geneva (12, 2020), 10.23731/CYRM-2020-0010.

[2] FCC collaboration, *FCC-ee: The Lepton Collider: Future Circular Collider Conceptual Design Report Volume 2*, *Eur. Phys. J. ST* **228** (2019) 261.

[3] FCC collaboration, *FCC-hh: The Hadron Collider: Future Circular Collider Conceptual Design Report Volume 3*, *Eur. Phys. J. ST* **228** (2019) 755.

[4] A. Buckley et al., *General-purpose event generators for LHC physics*, *Phys. Rept.* **504** (2011) 145 [1101.2599].

[5] M. Narain et al., *The Future of US Particle Physics - The Snowmass 2021 Energy Frontier Report*, 2211.11084.

[6] HEP Software Foundation collaboration, *A Roadmap for HEP Software and Computing R&D for the 2020s*, *Comput. Softw. Big Sci.* **3** (2019) 7 [1712.06982].

[7] HSF Physics Event Generator WG collaboration, *Challenges in Monte Carlo Event Generator Software for High-Luminosity LHC*, *Comput. Softw. Big Sci.* **5** (2021) 12 [2004.13687].

[8] European Strategy Group, *2020 Update of the European Strategy for Particle Physics*, CERN Council, Geneva (2020), 10.17181/ESU2020.

[9] M. van Beekveld, E. Bothmann, A. Buckley, C. Gütschow, P. Skands and R. Winterhalder, *The Monte Carlo Ecosystem in High-Energy Physics: A Primer*, 2605.16036.

[10] G.P. Lepage, *A New Algorithm for Adaptive Multidimensional Integration*, *J. Comput. Phys.* **27** (1978) 192.

[11] G.P. Lepage, *Adaptive multidimensional integration: VEGAS enhanced*, *J. Comput. Phys.* **439** (2021) 110386 [2009.05112].

[12] R. Kleiss and R. Pittau, *Weight optimization in multichannel Monte Carlo*, *Comput. Phys. Commun.* **83** (1994) 141 [hep-ph/9405257].

[13] T. Ohl, *Vegas revisited: Adaptive Monte Carlo integration beyond factorization*, *Comput. Phys. Commun.* **120** (1999) 13 [hep-ph/9806432].

[14] F. Maltoni and T. Stelzer, *MadEvent: Automatic event generation with MadGraph*, *JHEP* **02** (2003) 027 [hep-ph/0208156].

[15] F. Krauss, R. Kuhn and G. Soff, *AMEGIC++ 1.0: A Matrix element generator in C++*, *JHEP* **02** (2002) 044 [hep-ph/0109036].

[16] C.G. Papadopoulos, *PHEGAS: A Phase space generator for automatic cross-section computation*, *Comput. Phys. Commun.* **137** (2001) 247 [hep-ph/0007335].

[17] W. Kilian, T. Ohl and J. Reuter, *WHIZARD: Simulating Multi-Particle Processes at LHC and ILC*, *Eur. Phys. J. C* **71** (2011) 1742 [0708.4233].

[18] S. Jadach, *Foam: A General purpose cellular Monte Carlo event generator*, *Comput. Phys. Commun.* **152** (2003) 55 [physics/0203033].

[19] T. Gleisberg and S. Hoeche, *Comix, a new matrix element generator*, *JHEP* **12** (2008) 039 [0808.3674].

[20] Sherpa collaboration, *Event generation with Sherpa 3*, *JHEP* **12** (2024) 156 [2410.22148].

[21] S. Höche, S. Prestel and H. Schulz, *Simulation of Vector Boson Plus Many Jet Final States at the High Luminosity LHC*, *Phys. Rev. D* **100** (2019) 014024 [1905.05120].

[22] M.D. Klimek and M. Perelstein, *Neural Network-Based Approach to Phase Space Integration*, *SciPost Phys.* **9** (2020) 053 [1810.11509].

[23] E. Bothmann, T. Janßen, M. Knobbe, T. Schmale and S. Schumann, *Exploring phase space with Neural Importance Sampling*, *SciPost Phys.* **8** (2020) 069 [2001.05478].

[24] C. Gao, S. Höche, J. Isaacson, C. Krause and H. Schulz, *Event Generation with Normalizing Flows*, *Phys. Rev. D* **101** (2020) 076002 [2001.10028].

[25] B. Stienen and R. Verheyen, *Phase space sampling and inference from weighted events with autoregressive flows*, *SciPost Phys.* **10** (2021) 038 [2011.13445].

[26] T. Heimel, R. Winterhalder, A. Butter, J. Isaacson, C. Krause, F. Maltoni et al., *MadNIS - Neural multi-channel importance sampling*, *SciPost Phys.* **15** (2023) 141 [2212.06172].

[27] R. Verheyen, *Event Generation and Density Estimation with Surjective Normalizing Flows*, *SciPost Phys.* **13** (2022) 047 [2205.01697].

[28] T. Heimel, N. Huetsch, F. Maltoni, O. Mattelaer, T. Plehn and R. Winterhalder, *The MadNIS Reloaded*, *arXiv preprint* (2023) [2311.01548].

[29] T. Heimel, O. Mattelaer, T. Plehn and R. Winterhalder, *Differentiable MadNIS-Lite*, *SciPost Phys.* **18** (2025) 017 [2408.01486].

[30] H. Kharraziha and S. Moretti, *The Metropolis algorithm for on-shell four momentum phase space*, *Comput. Phys. Commun.* **127** (2000) 242 [hep-ph/9909313].

[31] S. Weinzierl, *A General algorithm to generate unweighted events for next-to-leading order calculations in electron positron annihilation*, *JHEP* **08** (2001) 028 [hep-ph/0106146].

[32] K. Kroeninger, S. Schumann and B. Willenberg, *(MC)**3 – a Multi-Channel Markov Chain Monte Carlo algorithm for phase-space sampling*, *Comput. Phys. Commun.* **186** (2015) 1 [1404.4328].

[33] S. La Cagnina, C. Grunwald, T. Janßen, K. Kröninger and S. Schumann, *Phase space sampling with Markov Chain Monte Carlo methods*, *EPJ Web Conf.* **337** (2025) 01319 [2412.12963].

[34] K. Danziger, T. Janßen, S. Schumann and F. Siegert, *Accelerating Monte Carlo event generation – rejection sampling using neural network event-weight estimates*, *SciPost Phys.* **12** (2022) 164 [2109.11964].

[35] T. Janßen, D. Maître, S. Schumann, F. Siegert and H. Truong, *Unweighting multijet event generation using factorisation-aware neural networks*, *SciPost Phys.* **15** (2023) 107 [2301.13562].

[36] T. Herrmann, T. Janßen, M. Schenker, S. Schumann and F. Siegert, *Accelerating multijet-merged event generation with neural network matrix element surrogates*, *SciPost Phys.* **20** (2026) 071 [2506.06203].

[37] R.M. Neal, *MCMC using Hamiltonian dynamics*, in *Handbook of Markov Chain Monte Carlo*, pp. 113–162, Chapman and Hall/CRC (2011) [1206.1901].

[38] M.D. Hoffman and A. Gelman, *The No-U-Turn Sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo*, *J. Mach. Learn. Res.* **15** (2014) 1593 [1111.4246].

[39] G.O. Roberts and R.L. Tweedie, *Exponential convergence of Langevin distributions and their discrete approximations*, *Bernoulli* **2** (1996) 341.

[40] X. Cheng, N.S. Chatterji, P.L. Bartlett and M.I. Jordan, *Underdamped Langevin MCMC: A non-asymptotic analysis*, in *Conference on Learning Theory*, pp. 300–323, PMLR, 2018 [1707.03663].

[41] N. Brosse, A. Durmus, É. Moulines and S. Sabanis, *The Tamed Unadjusted Langevin Algorithm*, *Stochastic Processes and their Applications* **129** (2019) 3638 [1710.05559].

[42] M. Girolami and B. Calderhead, *Riemann Manifold Langevin and Hamiltonian Monte Carlo Methods*, *Journal of the Royal Statistical Society Series B: Statistical Methodology* **73** (2011) 123.

[43] M.K. Titsias, *Optimal Preconditioning and Fisher Adaptive Langevin Sampling*, in *Advances in Neural Information Processing Systems*, vol. 36, 2023 [2305.14442].

[44] A. Gelman, W.R. Gilks and G.O. Roberts, *Weak Convergence and Optimal Scaling of Random Walk Metropolis Algorithms*, *The Annals of Applied Probability* **7** (1997) 110.

[45] C. Pasarica and A. Gelman, *Adaptively scaling the metropolis algorithm using expected squared jumped distance*, *Statistica Sinica* (2010) 343.

[46] H. Robbins and S. Monro, *A Stochastic Approximation Method*, *The Annals of Mathematical Statistics* (1951) 400.

[47] G.O. Roberts and J.S. Rosenthal, *Coupling and ergodicity of adaptive Markov chain Monte Carlo algorithms*, *Journal of Applied Probability* **44** (2007) 458.

[48] Y.-P. Hsieh, A. Kavis, P. Rolland and V. Cevher, *Mirrored Langevin Dynamics*, in *Advances in Neural Information Processing Systems*, vol. 31, 2018 [1802.10174].

[49] J. Gorham and L. Mackey, *Measuring Sample Quality with Stein's Method*, in *Advances in Neural Information Processing Systems*, vol. 28, 2015 [1506.03039].

[50] A. Anastasiou, A. Barp, F.-X. Briol, B. Ebner, R.E. Gaunt, F. Ghaderinezhad et al., *Stein's Method Meets Computational Statistics: A Review of Some Recent Developments*, *Statistical Science* **38** (2023) 120 [2105.03481].

[51] C. Stein, P. Diaconis, S. Holmes and G. Reinert, *Use of Exchangeable Pairs in the Analysis of Simulations*, *Lecture Notes-Monograph Series* (2004) 1.

[52] Q. Liu, J. Lee and M. Jordan, *A Kernelized Stein Discrepancy for Goodness-of-Fit Tests*, in *International Conference on Machine Learning*, pp. 276–284, PMLR, 2016 [1602.03253].

[53] K. Chwialkowski, H. Strathmann and A. Gretton, *A Kernel Test of Goodness of Fit*, in *International Conference on Machine Learning*, pp. 2606–2615, PMLR, 2016 [1602.02964].

[54] J. Gorham and L. Mackey, *Measuring Sample Quality with Kernels*, in *International Conference on Machine Learning*, pp. 1292–1301, PMLR, 2017 [1703.01717].

[55] T. Hu, Z. Chen, H. Sun, J. Bai, M. Ye and G. Cheng, *Stein Neural Sampler*, 1810.03545.

[56] W. Grathwohl, K.-C. Wang, J.-H. Jacobsen, D. Duvenaud and R. Zemel, *Learning the Stein Discrepancy for Training and Evaluating Energy-Based Models Without Sampling*, in *International Conference on Machine Learning*, pp. 3732–3747, PMLR, 2020 [2002.05616].

[57] S. Liu, T. Kanamori and D.J. Williams, *Estimating Density Models with Truncation Boundaries Using Score Matching*, *Journal of Machine Learning Research* **23** (2022) 1 [1910.03834].

[58] D.J. Williams and S. Liu, *Approximate Stein Classes for Truncated Density Estimation*, in *International Conference on Machine Learning*, pp. 37066–37090, PMLR, 2023 [2306.00602].

[59] W. Xu, *Standardisation-Function Kernel Stein Discrepancy: A Unifying View on Kernel Stein Discrepancy Tests for Goodness-of-Fit*, in *International Conference on Artificial Intelligence and Statistics*, pp. 1575–1597, PMLR, 2022.

[60] L. Heinrich and M. Kagan, *Differentiable Matrix Elements with MadJax*, *J. Phys. Conf. Ser.* **2438** (2023) 012137 [2203.00057].

[61] S. Carrazza, J. Cruz-Martinez, M. Rossi and M. Zaro, *MadFlow: towards the automation of Monte Carlo simulation on GPU for particle physics processes*, *EPJ Web Conf.* **251** (2021) 03022 [2105.10529].

[62] S. Carrazza, J. Cruz-Martinez, M. Rossi and M. Zaro, *MadFlow: automating Monte Carlo simulation on GPU for particle physics processes*, *Eur. Phys. J. C* **81** (2021) 656 [2106.10279].

[63] T. Heimel, O. Mattelaer and R. Winterhalder, *MadSpace – Event Generation for the Era of GPUs and ML*, 2602.06895.

[64] F. Bishara and M. Montull, *Machine learning amplitudes for faster event generation*, *Phys. Rev. D* **107** (2023) L071901 [1912.11055].

[65] S. Badger and J. Bullock, *Using neural networks for efficient evaluation of high multiplicity scattering amplitudes*, *JHEP* **06** (2020) 114 [2002.07516].

[66] J. Aylett-Bullock, S. Badger and R. Moodie, *Optimising simulations for diphoton production at hadron colliders using amplitude neural networks*, *JHEP* **08** (2021) 066 [2106.09474].

[67] D. Maître and H. Truong, *A factorisation-aware Matrix element emulator*, *JHEP* **11** (2021) 066 [2107.06625].

[68] S. Badger, A. Butter, M. Luchmann, S. Pitz and T. Plehn, *Loop amplitudes from precision networks*, *SciPost Phys. Core* **6** (2023) 034 [2206.14831].

[69] D. Maître and H. Truong, *One-loop matrix element emulation with factorisation awareness*, *JHEP* **05** (2023) 159 [2302.04005].

[70] H. Bahl, N. Elmer, L. Favaro, M. Haussmann, T. Plehn and R. Winterhalder, *Accurate surrogate amplitudes with calibrated uncertainties*, *SciPost Phys. Core* **8** (2025) 073 [2412.12069].

[71] J. Brehmer, V. Bresó, P. de Haan, T. Plehn, H. Qu, J. Spinner et al., *A Lorentz-equivariant transformer for all of the LHC*, *SciPost Phys.* **19** (2025) 108 [2411.00446].

[72] V. Bresó-Pla, G. Heinrich, V. Magerya and A. Olsson, *Interpolating amplitudes*, *SciPost Phys.* **19** (2025) 123 [2412.09534].

[73] H. Bahl, N. Elmer, T. Plehn and R. Winterhalder, *Amplitude Uncertainties Everywhere All at Once*, *SciPost Phys.* **20** (2026) 083 [2509.00155].

[74] L. Favaro, G. Gerhartz, F.A. Hamprecht, P. Lippmann, S. Pitz, T. Plehn et al., *Lorentz-Equivariance without Limitations*, 2508.14898.

[75] J.M. Villadamigo, R. Frederix, T. Plehn, T. Vitos and R. Winterhalder, *FASTColor – Full-color Amplitude Surrogate Toolkit for QCD*, 2509.07068.

[76] L. Beccatini, F. Maltoni, O. Mattelaer and R. Winterhalder, *Amplitude Surrogates for Multi-Jet Processes*, 2512.11036.

[77] H. Bahl, V. Bresó-Pla, A. Butter and J.I. Ramirez, *Scaling laws for amplitude surrogates*, 2601.13308.

[78] H. Bahl, J. Braun, G. Heinrich, T. Plehn and R. Revelli, *How to Trust Learned Loop Amplitudes*, 2601.00950.

[79] D.P. Kingma and J. Ba, *Adam: A Method for Stochastic Optimization*, in *International Conference on Learning Representations*, 2015 [1412.6980].

[80] S. Chewi, C. Lu, K. Ahn, X. Cheng, T. Le Gouic and P. Rigollet, *Optimal Dimension Dependence of the Metropolis-Adjusted Langevin Algorithm*, in *Conference on Learning Theory*, pp. 1260–1300, PMLR, 2021 [2012.12810].

[81] NNPDF collaboration, *The path to proton structure at 1% accuracy*, *Eur. Phys. J. C* **82** (2022) 428 [2109.02653].

[82] J. Bradbury, R. Frostig, P. Hawkins, M.J. Johnson, Y. Katariya, C. Leary et al., "JAX: composable transformations of Python+NumPy programs." https://github.com/jax-ml/jax, 2018.

[83] A. Buckley, J. Ferrando, S. Lloyd, K. Nordström, B. Page, M. Rüfenacht et al., *LHAPDF6: parton density access in the LHC precision era*, *Eur. Phys. J. C* **75** (2015) 132 [1412.7420].

[84] E. Bothmann, T. Childers, W. Giele, F. Herren, S. Hoeche, J. Isaacson et al., *Efficient phase-space generation for hadron collider event simulation*, *SciPost Phys.* **15** (2023) 169 [2302.10449].

[85] S. Catani, Y.L. Dokshitzer, M.H. Seymour and B.R. Webber, *Longitudinally invariant K_t clustering algorithms for hadron hadron collisions*, *Nucl. Phys. B* **406** (1993) 187.

[86] S.D. Ellis and D.E. Soper, *Successive combination jet algorithm for hadron collisions*, *Phys. Rev. D* **48** (1993) 3160 [hep-ph/9305266].

[87] W. Baur and V. Strassen, *The complexity of partial derivatives*, *Theoretical Computer Science* **22** (1983) 317.

[88] A. Griewank, *Who invented the reverse mode of differentiation?*, in *Optimization stories*, pp. 389–400, European Mathematical Society Publishing House (2012).